

ASESII 24A02 Project-Based Quiz 1

Ridge, Lasso, and Regularization for Neural Features

Group 06: Le Dat Phuc An_24125052-Pham Nguyen Minh Quan_24125041-Tran Canh Anh Tuan_241

2026-07-17

Contents

Authorship and contribution statement

We confirm that every group member can explain the submitted analysis. AI use is reported in Group06_AI_Log.csv, and the group checked every AI-assisted item used in this report.

Student	ID	Main contributions	Verification performed
Le Dat Phuc An	24125052	Sections 1, 2A, 2B, 5A, and 5B.	Verified Sections 2D, 3B, and 4C by reproducing results and reviewing the report.
Pham Nguyen Minh Quan	24125041	Sections 2D, 2E, 3A, 3B, and 4B.	Verified Sections 1, 2A, 5A, and 5B by checking methodology, code, and documentation.
Tran Canh Anh Tuan	24125048	Sections 2A,2B,3C, 4A, 4C, 4E, 5A.	Verified Sections 2B, 2C, and 4B by reviewing implementation and numerical results.
Vong Chi Van	24125049	Sections 2A,2B,2C, 4A, 4E.	Verified Sections 2E, 3A, and 3C by checking derivations, code, and final outputs.

Abstract

Maximum 150 words: prediction problem, methods, main evidence, and recommendation.

Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
       paste(missing_packages, collapse = ", "))
}

library(tidyverse)
library(glmnet)
library(broom)
library(knitr)

group_number <- as.integer(params$group_number)
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}

seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)
```

Shared helper functions

```
find_exam_data <- function(filename) {
  input <- knitr::current_input(dir = TRUE)
  input_dir <- if (length(input) && nzchar(input)) dirname(input) else getwd()
  candidates <- c(
    file.path(input_dir, "data", filename),
    file.path(input_dir, "..", "data", filename),
    file.path(getwd(), "data", filename)
  )
  existing <- candidates[file.exists(candidates)]
  if (!length(existing)) stop("Cannot locate ", filename, " by a relative path.")
  hits <- unique(normalizePath(existing, winslash = "/", mustWork = TRUE))
  if (length(hits) > 1L && length(unique(unname(tools::md5sum(hits)))) > 1L) {
    stop("Multiple nonidentical copies of ", filename, " were found.")
  }
  hits[[1L]]
}

split_rows <- function(n, train_prop = 0.80, seed) {
```

```

stopifnot(n >= 10L, train_prop > 0, train_prop < 1)
set.seed(seed)
train <- sort(sample.int(n, floor(train_prop * n), replace = FALSE))
list(train = train, test = setdiff(seq_len(n), train))
}

fit_scaler <- function(x, tol = 1e-12) {
  x <- as.matrix(x)
  center <- colMeans(x)
  centered <- sweep(x, 2L, center, "-")
  scale <- sqrt(colMeans(centered^2))
  keep <- is.finite(scale) & scale > tol
  if (!any(keep)) stop("No nonconstant training predictor remains.")
  list(
    center = center[keep],
    scale = scale[keep],
    keep = keep,
    dropped = colnames(x)[!keep]
  )
}

apply_scaler <- function(x, prep) {
  x <- as.matrix(x)[, prep$keep, drop = FALSE]
  x <- sweep(x, 2L, prep$center, "-")
  sweep(x, 2L, prep$scale, "/")
}

make_foldid <- function(n, k = 5L, seed) {
  if (k < 3L || k > n) stop("Require 3 <= k <= number of training rows.")
  set.seed(seed)
  sample(rep(seq_len(k), length.out = n))
}

fit_cv_glmnet <- function(x, y, alpha, foldid) {
  glmnet::cv.glmnet(
    x = x,
    y = y,
    family = "gaussian",
    alpha = alpha,
    foldid = foldid,
    standardize = FALSE,
    intercept = TRUE,
    type.measure = "mse"
  )
}

score_regression <- function(y, prediction) {

```

```

y <- as.numeric(y)
prediction <- as.numeric(prediction)
if (length(y) != length(prediction) || any(!is.finite(c(y, prediction)))) {
  stop("Metrics require equal-length finite vectors.")
}
c(RMSE = sqrt(mean((y - prediction)^2)),
  MAE = mean(abs(y - prediction)))
}

count_nonzero <- function(fit, s = "lambda.min", tol = 1e-8) {
  b <- as.matrix(stats::coef(fit, s = s))
  slopes <- b[rownames(b) != "(Intercept)", 1L]
  sum(abs(slopes) > tol)
}

safe_condition_numbers <- function(x, lambda, tol = 1e-10) {
  G <- crossprod(x) / nrow(x)
  eigenvalues <- eigen(G, symmetric = TRUE, only.values = TRUE)$values
  largest <- max(eigenvalues)
  smallest <- max(min(eigenvalues), 0)
  cutoff <- tol * max(1, largest)
  c(
    gram = if (smallest <= cutoff) Inf else largest / smallest,
    ridge = (largest + lambda) / (smallest + lambda)
  )
}

plot_model_result <- function(model, model_name) {
  model_result <- augment(model)
  ggplot(model_result, aes(x = .fitted, y = .resid)) +
    geom_point(color = "steelblue", size = 2, alpha = 0.8) +

    geom_hline(yintercept = 0, linetype = "dashed", color = "red", linewidth = 1) +

    theme_minimal() +

    labs(
      title = paste("Residual Plot Model:", model_name),
      subtitle = "Checking for non-linear patterns and constant variance",
      x = "Predicted lpsa (.fitted)",
      y = "Residuals (.resid)"
    )
}

vif_manual <- function(model) {
  X <- model.matrix(model)[, -1, drop = FALSE]

  sapply(seq_len(ncol(X)), function(i) {

```

```

    fit <- lm(X[, i] ~ X[, -i])
    1 / (1 - summary(fit)$r.squared)
  }) |> setNames(colnames(X))
}

coef_norm <- function(coefs) {
  b <- as.matrix(coefs)
  sqrt(sum(b[-1, ]^2))
}

nonzero_slopes <- function(coefs, tol = 1e-8) {
  b <- as.matrix(coefs)
  sum(abs(b[-1, ]) > tol)
}

```

1 Problem 1. Prediction design and leakage control

1.1 1A. Target and predictors

```

config <- if (group_number %% 2L == 1L) {
  list(
    file = "fat.csv",
    response = "brozek",
    excluded = c("brozek", "siri", "density", "free")
  )
} else {
  list(
    file = "prostate.csv",
    response = "lpsa",
    excluded = "lpsa"
  )
}

data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)
predictors <- setdiff(names(data_raw), config$excluded)
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]

if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {
  stop("The assigned response and predictors must be numeric.")
}

```

Define the prediction task and justify every exclusion.

1.1.1 1. Response = *lpsa*

lpsa measures the natural Logarithm of Prostate-Specific Antigen, which is a continuous number

1.1.2 2. Candidate Predictors

Candidate Predictors	Full Name	Type
lcavol	Log Cancer Volume	Continuous
lweight	Log Prostate Weight	Continuous
age	Age	Continuous
lbph	Log Benign Prostatic Hyperplasia	Continuous
svi	Seminal Vesicle Invasion	Binary (0 / 1)
lcp	Log Capsular Penetration	Continuous
gleason	Gleason Score	Ordered Categorical
pgg45	Percentage Gleason Grade 4/5	Continuous (0–100)

1.1.3 3. Unit of Observation

Each row of the dataset contains the information of a radical prostatectomy treated patient which comprises of the target and the predictors values.

1.1.4 4. Study Population

The dataset contains 97 patients aged 41 to 79 with clinically localized adenocarcinoma who went through radical prostatectomy at Stanford University Center by Dr. Thomas A. Stamey and his surgical team during the late 1980s.

1.1.5 5. Prediction Objective

To build a regression model that predicts the log prostate-specific antigen level (lpsa) using the clinical variables (lcavol, lweight, age, lbph, svi, lcp, gleason, and pgg45).

1.1.6 6. Dataset Source

<https://sci-hub.sg/storage/2024/6429/4cbb75646440af614abced23a70ba059/stamey1989.pdf>

1.1.7 7. Required Exclusions

The response variable, `lpsa`, was excluded from the predictor matrix because it is the outcome to be predicted. The remaining eight clinical variables were retained as predictors. Since the dataset contains no variables derived from `lpsa`, no additional exclusions were required. This prevents target leakage and ensures a fair comparison among the regression models.

1.2 1B. Split and train-only preprocessing

```
split <- split_rows(nrow(analysis_data), seed = seeds$split)
train_id <- split$train
test_id <- split$test

x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])
```

```
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test  <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)
y_train <- y_raw[train_id]
y_test  <- y_raw[test_id]

foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
```

Report seeds, dimensions, scaling, folds, and leakage safeguards.

1.2.1 1. Split seed

- Seed split for 80/20 split: 240206
- Seed cv for K-fold split: 240306

1.2.2 2. Train/Test Split

The dataset of 97 observations was partitioned into:

- **Training Set:** 77 observations
- **Test Set:** 20 observations

1.2.3 3. Feature Scaling Rule

Continuous predictor variables were centered and scaled using the helper functions `fit_scaler()` and `apply_scaler()`. The scaling parameters were estimated **only from the training data** in `fit_scaler()` and then applied unchanged to both the training and test sets by `apply_scaler()`.

Specifically, each predictor was transformed as

$$x_{\text{scaled}} = \frac{x - \bar{x}_{\text{train}}}{\sqrt{\frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} (x_i - \bar{x}_{\text{train}})^2}},$$

where $\bar{x}_{\text{train}}$ is the mean of the predictor in the training set, and the denominator is the population standard deviation computed from the training observations. Using the same scaling parameters for the test set prevents information leakage from the holdout data.

1.2.4 4. Five-Fold Assignment Rule

A single, immutable five-fold cross-validation assignment vector (`foldid`) was generated on the 77 training observations using the designated `cv` random seed. To ensure fair and controlled comparisons, this exact same fold assignment was reused across all models evaluated:

- Ridge Regression
- Lasso Regression
- Elastic Net
- All subsequent hyperparameter grid evaluations

1.2.5 5. Missing-Value Plan

No missing values were detected. If any had been present, they would have been handled using a training-only imputation procedure before model fitting.

1.3 1C. Training-data exploration

The exploratory analysis in this section uses only the training observations to avoid information leakage from the holdout test set.

1.3.1 Response distribution

```
summary(y_train)
sd(y_train)

hist(
  y_train,
  main = "Histogram of y_train",
  xlab = "y_train"
)

qqnorm(y_train)
qqline(y_train)
```

The response variable (`lpsa`) has a mean of **2.54** and a median of **2.59**, indicating an approximately symmetric distribution. Most observations lie between **1.73** (first quartile) and **3.28** (third quartile), with a standard deviation of **1.15**.

The histogram suggests that most observations are concentrated around the center, while only a few observations appear in the lower and upper tails. The normal Q-Q plot shows that most observations follow the reference line reasonably well, with slight departures in the tails. Overall, the response distribution is approximately normal, making it suitable for linear regression.

1.3.2 Relationship exploration

```
pairs(
  analysis_data[train_id, ],
  pch = 20,
  cex = 0.6
)
```

1.3.2.1 Scatterplot matrix The scatterplot matrix provides a visual overview of pairwise relationships among the response and predictors. Several predictors exhibit approximately linear relationships with the response, particularly `lcavol`. Strong positive associations are also visible between several predictor pairs, suggesting that some variables may contain overlapping information.

```
cor_matrix <- cor(analysis_data[train_id, ])

heatmap_df <- expand_grid(
  Var1 = rownames(cor_matrix),
  Var2 = colnames(cor_matrix)
)
```

```
heatmap_df$value <- c(cor_matrix)

ggplot(heatmap_df, aes(Var1, Var2, fill = value)) +
  geom_tile() +
  geom_text(aes(label = round(value, 2)), size = 3) +
  scale_fill_gradient2(
    low = "blue",
    mid = "white",
    high = "red",
    midpoint = 0,
    limits = c(-1, 1)
  ) +
  coord_fixed() +
  theme_minimal()
```

1.3.2.2 Correlation matrix The correlation heatmap shows that **lcavol** has the strongest linear relationship with the response ($r = 0.75$), indicating that tumor volume is the single most informative predictor of **lpsa**. Moderate positive correlations are also observed for **lcp** ($r = 0.59$) and **svi** ($r = 0.58$), whereas **age**, **lbph**, and **lweight** exhibit relatively weak marginal correlations with the response.

1.3.3 Potential multicollinearity

The correlation matrix indicates several moderately to strongly correlated predictor pairs.

The strongest pairwise correlations include

- **gleason** and **pgg45** ($r = 0.76$),
- **lcavol** and **lcp** ($r = 0.73$),
- **lcp** and **svi** ($r = 0.67$),
- **lcavol** and **svi** ($r = 0.59$),
- **lcavol** and **gleason** ($r = 0.47$).

These relationships suggest that some predictors may contain overlapping information. However, pairwise correlation alone does not imply severe multicollinearity, since multicollinearity depends on the joint linear relationship among all predictors rather than individual pairs. Therefore, formal diagnostics such as the variance inflation factor (VIF) will be performed after fitting the OLS model to determine whether coefficient instability is present.

1.3.4 One possible anomaly or influential observation

```
boxplot(analysis_data[train_id, ])
```

The boxplots identify several potential outliers, particularly in **age** and **pgg45**. Among the predictors, **pgg45** exhibits the largest variability, while **age** contains several observations outside the whiskers. These observations are retained because there is no evidence that they result from data entry errors.

1.3.5 Question for regularized models

Based on the exploratory analysis, the following question will guide the remaining analysis:

Can regularization improve predictive performance and coefficient stability when predictors contain overlapping information?

Interpret multicollinearity, one anomaly, and the question regularization will address.

2 Problem 2. OLS, ridge, and lasso

2.1 2A. OLS baseline

The ordinary least squares (OLS) model was fitted using only the standardized training observations. Throughout this section, the holdout test set remained completely untouched to ensure a fair comparison with the regularized regression models.

2.1.1 Fit OLS

```
x_train_df <- as.data.frame(x_train)
x_train_df$lpsa <- y_train

ols_model <- lm(lpsa ~ ., data = x_train_df)
```

The fitted OLS model serves as the baseline model for all subsequent comparisons with ridge and lasso regression.

2.1.2 Coefficient estimates

```
summary(ols_model)
```

The fitted OLS model is statistically significant overall ($F = 15.78$, $p < 0.001$), indicating that at least one predictor contributes to explaining the variation in the response variable.

The model achieves a **Multiple R^2** of 0.6500 and an **Adjusted R^2** of 0.6088, suggesting that approximately **65%** of the variability in `lpsa` is explained by the eight clinical predictors.

Among the predictors,

- **lcavol** ($\beta = 0.674$, $p < 0.001$),
- **lweight** ($\beta = 0.202$, $p = 0.0438$),
- **age** ($\beta = -0.194$, $p = 0.0435$), and
- **svi** ($\beta = 0.282$, $p = 0.0206$)

are statistically significant at the 5% significance level.

The remaining predictors (`lbph`, `lcp`, `gleason`, and `pgg45`) are not individually significant after adjusting for the other predictors in the model.

The estimated coefficient for **age** is negative ($\beta = -0.194$). This result suggests that the estimated effect of **age** changes after accounting for the remaining predictors in the model. The possibility of correlation among predictors is therefore investigated using variance inflation factors in the following subsection.

2.1.3 Training error

```
train_pred <- predict(ols_model)

train_mse <- mean((y_train - train_pred)^2)
train_rmse <- sqrt(train_mse)

cat("Training MSE :", round(train_mse, 4), "\n")
cat("Training RMSE:", round(train_rmse, 4), "\n")
```

The fitted OLS model achieved

- Training MSE = 0.4544
- Training RMSE = 0.6741

These quantities measure the goodness-of-fit on the training observations used to estimate the regression coefficients. Since the model is evaluated on the same data used for fitting, the training error is expected to underestimate the prediction error on unseen observations.

2.1.4 Five-fold cross-validation

```
cv_mse <- numeric(5)
coefs_list <- list()
for(i in 1:5){

  train_fold <- x_train_df[foldid != i, ]
  valid_fold <- x_train_df[foldid == i, ]

  model <- lm(lpsa ~ ., data = train_fold)

  pred <- predict(model, newdata = valid_fold)
  fold_coef <- as.matrix(coef(model, s="lambda.min"))
  coefs_list[[i]] <- as.data.frame(t(fold_coef))
  cv_mse[i] <- mean((valid_fold$lpsa - pred)^2)
}
coef_ols_df <- bind_rows(coefs_list)
cv_table <- data.frame(
  Fold = 1:5,
  Validation_MSE = round(cv_mse, 4)
)

knitr::kable(cv_table)
```

Fold	Validation_MSE
1	0.8259
2	0.8568
3	0.4011
4	0.4893

Fold	Validation_MSE
5	0.9002

```
cat("Mean CV MSE :", round(mean(cv_mse), 4), "\n")

## Mean CV MSE : 0.6947

cat("Mean CV RMSE:", round(sqrt(mean(cv_mse)), 4), "\n")

## Mean CV RMSE: 0.8335
```

The validation errors obtained from the five folds are

Fold	Validation MSE
1	0.8259
2	0.8568
3	0.4011
4	0.4893
5	0.9002

The average cross-validation error is

- **Mean CV MSE = 0.6947**
- **Mean CV RMSE = 0.8335**

As expected, the cross-validation error is larger than the training error because each validation fold consists of observations that were not used to estimate the regression coefficients. Although some variation exists among the five folds, the validation errors remain within a similar range, suggesting that the predictive performance of the OLS model is reasonably stable across different training subsets.

2.1.5 Residual diagnostics

```
plot(
  fitted(ols_model),
  resid(ols_model),
  xlab = "Fitted values",
  ylab = "Residuals",
  main = "Residuals vs Fitted"
)

abline(h = 0, col = "red")
```

2.1.5.1 Residual versus fitted values The residual-versus-fitted plot shows that the residuals are randomly scattered around zero without noticeable curvature or funnel-shaped patterns. This suggests that the assumptions of

- linearity, and

- constant error variance (homoscedasticity)

are reasonably satisfied. No obvious systematic pattern indicating model misspecification is observed.

```
qqnorm(resid(ols_model))
qqline(resid(ols_model), col = "red")
```

2.1.5.2 Normal Q-Q plot The normal Q-Q plot indicates that most residuals lie close to the reference line, with only slight departures in the two tails. Therefore, the residual distribution appears approximately normal.

The normal Q-Q plot indicates that the residuals are approximately normally distributed, with only slight deviations in the tails. This behavior is consistent with the mildly skewed response distribution observed during the exploratory analysis. Since the residuals satisfy

$$\hat{\mathbf{e}} = (I - H)\mathbf{y},$$

where $\hat{\mathbf{e}}$ is the residual vector, $\mathbf{y}$ is the response vector, I is the identity matrix, and $H = X(X^\top X)^{-1}X^\top$ is the hat matrix, their distribution naturally reflects the response after removing the fitted linear component. Overall, there is no strong evidence against the normality assumption.

```
plot(ols_model, which = 5)
```

2.1.5.3 Influence plot Most observations have low leverage and moderate residuals. Although a few observations (e.g., 23, 37, and 54) stand out slightly, none appears to have an unusually large influence on the fitted model.

```
cook <- cooks.distance(ols_model)

plot(
  cook,
  type = "h",
  lwd = 3,
  col = "red",
  xlab = "Observation",
  ylab = "Cook's Distance",
  main = "Cook's Distance"
)

abline(
  h = 4 / nrow(x_train_df),
  col = "blue",
  lty = 2
)
```

2.1.5.4 Cook’s Distance Several observations exceed the reference line $4/n$, but all Cook’s distance values are well below 1. Therefore, there is no strong evidence of influential observations.

Taken together, the residual diagnostics do not reveal any serious violation of the standard assumptions underlying ordinary least squares regression.

2.1.6 Ill-conditioned design

```
vif_manual(ols_model)
```

The variance inflation factors (VIFs) were computed to assess multicollinearity among the predictors.

The VIF values range from **1.33** to **3.24**, with the largest values observed for

- lcp (3.24),
- pgg45 (2.97),
- gleason (2.68), and
- lcavol (2.42).

None of the predictors exceeds the commonly used threshold of **5**, indicating that severe multicollinearity is not present in the training design matrix.

This finding is consistent with the correlation analysis presented in Section 1C. Although several predictor pairs, such as **gleason–pgg45** and **lcavol–lcp**, exhibit moderate pairwise correlations, pairwise correlation alone does not necessarily imply problematic multicollinearity. The VIF measures how well each predictor can be explained by **all remaining predictors simultaneously**, making it a more appropriate diagnostic for coefficient instability.

Overall, the VIF results suggest that the correlations among predictors are not sufficiently strong to produce substantial inflation of the coefficient variances. Therefore, no clear evidence of an ill-conditioned design was found, and the OLS estimates provide an appropriate baseline for comparison with ridge and lasso regression in the following sections.

2.2 2B. Ridge

2.2.1 Fitting Ridge

The Ridge model is fitted using `glmnet` with `alpha = 0` on the training data.

```
ridge_model <- fit_cv_glmnet(
  x_train,
  y_train,
  alpha = 0,
  foldid = foldid
)

plot(ridge_model)

abline(v = -log(ridge_model$lambda.min),
       col = "green",
       lty = 2,
       lwd = 1.5)
```

```
abline(v = -log(ridge_model$lambda.1se),
       col = "blue",
       lty = 2,
       lwd = 1.5)

y_pos <- max(ridge_model$cvup)

text(-log(ridge_model$lambda.min),
     y_pos,
     "lambda.min",
     pos = 2,
     col = "green",
     cex = 0.9)

text(-log(ridge_model$lambda.1se),
     y_pos,
     "lambda.1se",
     pos = 2,
     col = "blue",
     cex = 0.9)
```

ASESII_24A02_ProjectExam_SampleSubmission_files/ASESII_24A02_ProjectExam_SampleSubmission_file

The cross-validation curve shows how the prediction error changes with the penalty parameter. The mean cross-validation error decreases until reaching its minimum at **lambda.min**. Around the minimum, the error changes only slightly, indicating that a range of penalty values provides similar predictive performance.

Because Ridge regression uses an L2 penalty, every coefficient is continuously shrunk toward zero as the penalty increases. However, Ridge does not apply coordinate-wise thresholding, so no coefficient is set exactly to zero and all eight predictors remain in the model.

2.2.2 Investigation on λ_{min} and λ_{1se}

The optimal penalty values selected by cross-validation are reported below.

```
cat(sprintf("lambda.min: %.5f\n", ridge_model$lambda.min))

## lambda.min: 0.28468

cat(sprintf("lambda.1se: %.5f\n\n", ridge_model$lambda.1se))

## lambda.1se: 1.66740
```

lambda.min is the value of λ that minimizes the mean cross-validation error. It is selected when predictive accuracy is the primary objective.

`lambda.1se` is the largest value of λ whose cross-validation error is within one standard error of the minimum. It applies stronger regularization, producing a simpler and more stable model while usually maintaining similar predictive performance.

The coefficient estimates and corresponding cross-validation error at `lambda.min` are shown below.

```
min_id <- which(ridge_model$lambda == ridge_model$lambda.min)

cat(sprintf("cvm.min: %.5f\n", ridge_model$cvm[min_id]))

## cvm.min: 0.63753

coef(ridge_model, s = "lambda.min")
```

```
## 9 x 1 sparse Matrix of class "dgCMatrix"
##           lambda.min
## (Intercept) 2.54491818
## lcavol      0.45872792
## lweight     0.17492397
## age         -0.11231149
## lbph        0.10203528
## svi         0.23691430
## lcp         0.08238970
## gleason     0.02447306
## pgg45       0.10953781
```

At `lambda.min`, the model achieves the lowest cross-validation error. The coefficients are moderately shrunk compared with the OLS estimates, while the most important predictors still retain relatively large coefficients.

The corresponding results for `lambda.1se` are shown below.

```
se1_id <- which(ridge_model$lambda == ridge_model$lambda.1se)

cat(sprintf("cvm.1se: %.5f\n\n", ridge_model$cvm[se1_id]))

## cvm.1se: 0.71834

coef(ridge_model, s = "lambda.1se")
```

```
## 9 x 1 sparse Matrix of class "dgCMatrix"
##           lambda.1se
## (Intercept) 2.54491818
## lcavol      0.23622160
## lweight     0.10299748
## age         -0.02250727
## lbph        0.05203722
## svi         0.15871610
## lcp         0.12657524
## gleason     0.04952838
## pgg45       0.08822978
```

Using the larger penalty `lambda.1se` shrinks every coefficient further toward zero than `lambda.min`.

The cross-validation error increases only slightly from , indicating that a more regularized model can achieve nearly the same predictive performance.

Unlike Lasso, Ridge regression continuously shrinks the coefficients but does not set any coefficient exactly to zero. Therefore, every predictor remains in the fitted model under both tuning parameter choices.

2.2.3 Coefficient paths

The coefficient paths show how the Ridge coefficients change as the penalty varies.

```
plot(ridge_model$glmnet.fit,
     xvar = "lambda",
     label = TRUE)

title("Ridge Coefficient Paths", line = 2.5)

abline(v = -log(ridge_model$lambda.min),
       col = "green",
       lty = 2,
       lwd = 1.5)

abline(v = -log(ridge_model$lambda.1se),
       col = "blue",
       lty = 2,
       lwd = 1.5)

legend("topleft",
      legend = c("lambda.min", "lambda.1se"),
      col = c("green", "blue"),
      lty = 2,
      lwd = 1.5,
      cex = 0.8,
      bg = "white")

predictor_names <- rownames(ridge_model$glmnet.fit$beta)

legend("topleft",
      inset = c(0, 0.21),
      legend = paste(1:length(predictor_names),
                     predictor_names,
                     sep = " - "),
      col = 1:length(predictor_names),
      lty = 1,
      lwd = 1.5,
      cex = 0.8,
      bg = "white",
      ncol = 2)
```

As the penalty decreases (moving from left to right), the coefficients move smoothly away from near zero toward their OLS estimates. Larger penalties produce stronger shrinkage, whereas smaller penalties allow the coefficients to approach the unregularized solution.

Unlike Lasso, Ridge continuously shrinks every coefficient and therefore does not produce exact zero coefficients.

2.2.4 General Behavior of Correlated Predictors Under Ridge Regression

Ridge regression is particularly effective when predictors are highly correlated. In ordinary least squares, correlated predictors compete to explain the same variation in the response, leading to unstable coefficient estimates with high variance.

Ridge addresses this issue by shrinking the coefficients of correlated predictors towards each other rather than allowing one predictor to dominate the others. This reduces the variability of the coefficient estimates and improves model stability while retaining all predictors in the model. This behaviour is reflected in the coefficient paths, where all coefficients are smoothly shrunk towards zero as the penalty increases.

Unlike Lasso, Ridge regression does not perform variable selection. Instead, all coefficients are continuously shrunk towards zero, but none becomes exactly zero.

2.2.4.1 Examples from the Data The effect of Ridge can be seen in several pairs of highly correlated predictors.

- **lcavol and lcp (correlation = 0.73):** Under OLS, `lcavol` receives a much larger positive coefficient while `lcp` is assigned a small negative coefficient. Ridge shrinks both coefficients, reducing this imbalance and allowing the two correlated predictors to share their predictive contribution.
- **pgg45 and gleason (correlation = 0.76):** OLS assigns substantially different coefficients to these correlated predictors. Ridge shrinks both estimates towards zero, producing more stable coefficient estimates while retaining both variables in the model.

2.3 2C. Lasso

2.3.1 Fitting Lasso

The lasso model is fitted using `glmnet` by setting `alpha = 1` on the training data.

```
lasso_model <- fit_cv_glmnet(
  x_train,
  y_train,
  alpha = 1,
  foldid = foldid
)
```

```

plot(lasso_model)

abline(v = -log(lasso_model$lambda.min),
       col = "green", lty = 2, lwd = 1.5)

abline(v = -log(lasso_model$lambda.1se),
       col = "blue", lty = 2, lwd = 1.5)

y_pos <- max(lasso_model$cvup)

text(-log(lasso_model$lambda.min), y_pos,
     labels = "lambda.min",
     pos = 2, col = "green", cex = 0.9)

text(-log(lasso_model$lambda.1se), y_pos,
     labels = "lambda.1se",
     pos = 2, col = "blue", cex = 0.9)

```

ASESII_24A02_ProjectExam_SampleSubmission_files/ASESII_24A02_ProjectExam_SampleSubmission_files

Unlike Ridge regression, Lasso performs both coefficient shrinkage and variable selection. As the penalty increases, some coefficients are shrunk exactly to zero, producing a simpler and more interpretable model. The cross-validation curve identifies the penalty giving the lowest prediction error ($\lambda_{\min}$) together with the largest penalty within one standard error (λ_{1se}).

2.3.2 Investigation on $\lambda_{\min}$ and λ_{1se}

The optimal penalties are

```

cat(sprintf("lambda.min: %.5f\n", lasso_model$lambda.min))

## lambda.min: 0.10970

cat(sprintf("lambda.1se: %.5f\n\n", lasso_model$lambda.1se))

## lambda.1se: 0.33502

```

The corresponding training cross-validation errors are

```

min_id <- which(lasso_model$lambda == lasso_model$lambda.min)
se1_id <- which(lasso_model$lambda == lasso_model$lambda.1se)

cat(sprintf("cvm.min : %.5f\n",
           lasso_model$cvm[min_id]))

## cvm.min : 0.63210

```

```
cat(sprintf("cvm.1se : %.5f\n\n",
           lasso_model$cvm[se1_id]))
```

```
## cvm.1se : 0.72788
```

The selected predictors under the two tuning rules are

```
cat("--- Nonzero coefficients (lambda.min) ---\n")
```

```
## --- Nonzero coefficients (lambda.min) ---
```

```
coef_min <- coef(lasso_model, s = "lambda.min")
print(coef_min[coef_min[, 1] != 0, ])
```

```
## (Intercept)      lcavol      lweight      svi      pgg45
## 2.54491818 0.60702254 0.10812626 0.17708356 0.02150988
```

```
cat("\n--- Nonzero coefficients (lambda.1se) ---\n")
```

```
##
```

```
## --- Nonzero coefficients (lambda.1se) ---
```

```
coef_1se <- coef(lasso_model, s = "lambda.1se")
print(coef_1se[coef_1se[, 1] != 0, ])
```

```
## (Intercept)      lcavol      svi
## 2.54491818 0.49176838 0.03845547
```

`lambda.min` prioritizes predictive accuracy and therefore usually keeps more predictors. `lambda.1se` accepts a small increase in prediction error in exchange for a sparser and easier-to-interpret model.

2.3.3 Coefficient paths

The coefficient paths illustrate how predictors enter and leave the model as the penalty changes.

```
plot(lasso_model$glmnet.fit,
     xvar = "lambda",
     label = TRUE)

title("Lasso Coefficient Paths", line = 2.5)

abline(v = -log(lasso_model$lambda.min),
       col = "green",
       lty = 2,
       lwd = 1.5)

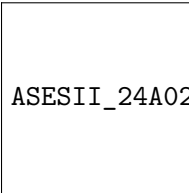
abline(v = -log(lasso_model$lambda.1se),
       col = "blue",
       lty = 2,
       lwd = 1.5)

legend("topleft",
```

```

legend = c("lambda.min", "lambda.1se"),
col = c("green", "blue"),
lty = 2,
lwd = 1.5,
cex = 0.75,
bg = "white")

```



ASESII_24A02_ProjectExam_SampleSubmission_files/ASESII_24A02_ProjectExam_SampleSubmission_files

Unlike Ridge regression, Lasso coefficient paths frequently terminate at exactly zero. Variables that reach zero are removed from the model, while only the strongest predictors remain active under larger penalty values.

2.3.4 Application-specific interpretation

Lasso identifies the predictors that provide the strongest independent contribution to explaining `lpsa` after accounting for the remaining variables.

At `lambda.min`, four predictors remain active: `lcavol`, `lweight`, `svi`, and `pgg45`. Among these, `lcavol` has by far the largest coefficient (0.607), indicating that tumor volume is the strongest predictor of prostate-specific antigen levels. `svi` also remains, suggesting that seminal vesicle invasion contributes additional predictive information beyond tumor volume. `lweight` and `pgg45` have smaller coefficients but still improve predictive performance enough to be retained.

At `lambda.1se`, the stronger penalty removes both `lweight` and `pgg45`, leaving only `lcavol` and `svi`. This indicates that these two variables contain the most robust independent signal in the data. The cross-validation error increases only slightly (0.632 to 0.728), while the model is reduced from four predictors to two, yielding a much simpler and more interpretable model.

The variables excluded by Lasso (`age`, `lbph`, `lcp`, `gleason`, `lweight`, and `pgg45` under `lambda.1se`) are not necessarily unrelated to prostate cancer. Rather, after accounting for `lcavol` and `svi`, their remaining predictive contribution is too small to justify the additional model complexity. Some of their information is likely shared with the retained predictors because several clinical measurements are moderately correlated.

2.4 2D. Training-based comparison and locked core model

This section compares the candidate models using only the training data and cross-validation results. The holdout test set is not used so that model selection remains unbiased.

2.4.1 Training-based comparison

```

# OLS
cv_ols <- mean(cv_mse)
coef_ols <- coef(ols_model)
norm_ols <- coef_norm(coef_ols)

```

```

nz_ols <- nonzero_slopes(coef_ols)

# Ridge
coef_ridge_min <- coef(ridge_model, s = "lambda.min")
coef_ridge_1se <- coef(ridge_model, s = "lambda.1se")

id_min <- which.min(abs(ridge_model$lambda - ridge_model$lambda.min))
id_1se <- which.min(abs(ridge_model$lambda - ridge_model$lambda.1se))

cv_ridge_min <- ridge_model$cvm[id_min]
cv_ridge_1se <- ridge_model$cvm[id_1se]

norm_ridge_min <- coef_norm(coef_ridge_min)
norm_ridge_1se <- coef_norm(coef_ridge_1se)

nz_ridge_min <- nonzero_slopes(coef_ridge_min)
nz_ridge_1se <- nonzero_slopes(coef_ridge_1se)

# Lasso
coef_lasso_min <- coef(lasso_model, s = "lambda.min")
coef_lasso_1se <- coef(lasso_model, s = "lambda.1se")

id_min <- which.min(abs(lasso_model$lambda - lasso_model$lambda.min))
id_1se <- which.min(abs(lasso_model$lambda - lasso_model$lambda.1se))

cv_lasso_min <- lasso_model$cvm[id_min]
cv_lasso_1se <- lasso_model$cvm[id_1se]

norm_lasso_min <- coef_norm(coef_lasso_min)
norm_lasso_1se <- coef_norm(coef_lasso_1se)

nz_lasso_min <- nonzero_slopes(coef_lasso_min)
nz_lasso_1se <- nonzero_slopes(coef_lasso_1se)

comparison_table <- data.frame(
  Model = c(
    "OLS",
    "Ridge",
    "Ridge",
    "Lasso",
    "Lasso"
  ),
  Tuning = c(
    "None",
    "lambda.min",
    "lambda.1se",
    "lambda.min",
    "lambda.min"
  )
)

```

```

    "lambda.1se"
  ),
  CV_MSE = c(
    cv_ols,
    cv_ridge_min,
    cv_ridge_1se,
    cv_lasso_min,
    cv_lasso_1se
  ),
  Coefficient_Norm = c(
    norm_ols,
    norm_ridge_min,
    norm_ridge_1se,
    norm_lasso_min,
    norm_lasso_1se
  ),
  Nonzero_Slopes = c(
    nz_ols,
    nz_ridge_min,
    nz_ridge_1se,
    nz_lasso_min,
    nz_lasso_1se
  )
)
)

knitr::kable(comparison_table, digits = 4)

```

Model	Tuning	CV_MSE	Coefficient_Norm	Nonzero_Slopes
OLS	None	0.6947	0.8154	8
Ridge	lambda.min	0.6375	0.5827	8
Ridge	lambda.1se	0.7183	0.3480	8
Lasso	lambda.min	0.6321	0.6419	4
Lasso	lambda.1se	0.7279	0.4933	2

Regularization improves predictive performance over OLS, with both Ridge (`lambda.min`) and Lasso (`lambda.min`) achieving lower cross-validation errors. Ridge shrinks the coefficients but retains all eight predictors, whereas Lasso also performs variable selection. Lasso (`lambda.min`) achieves the lowest cross-validation error while retaining only four predictors, providing the best balance between prediction accuracy and model simplicity.

2.4.2 Core model nominated before holdout evaluation

Based on the training results, **Lasso with `lambda.min`** is selected as the core candidate. It achieves the lowest cross-validation error while reducing the model from eight predictors to four. Although Ridge (`lambda.min`) has similar predictive performance, it retains all predictors and is therefore less interpretable.

2.4.3 Why `lambda.min` and `lambda.1se` answer different decision questions

`lambda.min` prioritizes predictive performance by minimizing the cross-validation error. In contrast, `lambda.1se` selects a larger penalty to obtain a simpler and more stable model while accepting a small increase in prediction error.

2.4.4 Predictive performance versus interpretability

Predictive performance is assessed by the cross-validation error, whereas interpretability depends on model simplicity. In this analysis, Lasso (`lambda.min`) achieves the lowest prediction error while retaining only four predictors, making it the preferred model before the final holdout evaluation.

2.5 2E. Perturbation or stability check

This stability check investigates how Ridge and Lasso respond to severe multicollinearity by duplicating a predictor with a small amount of random noise. The experiment uses only the training data.

2.5.1 Prediction before running the check

Before running the perturbation, we expect Ridge to distribute the coefficient between the original and duplicated predictors because the ridge penalty encourages correlated predictors to share their effects. In contrast, Lasso is expected to retain only one of the two predictors while shrinking the other coefficient to zero.

```
# Duplicate a strong predictor with small random noise
target_idx <- 1
target_name <- colnames(x_train)[target_idx]

duplicate_var <- x_train[, target_idx] +
  rnorm(nrow(x_train), sd = 0.01)

x_train_pert <- cbind(
  x_train,
  duplicate_var = duplicate_var
)

# Refit Ridge and Lasso using the perturbed data
ridge_pert <- fit_cv_glmnet(
  x_train_pert,
  y_train,
  alpha = 0,
  foldid = foldid
)

lasso_pert <- fit_cv_glmnet(
  x_train_pert,
  y_train,
  alpha = 1,
  foldid = foldid
)
```

```

)

ridge_coef <- coef(ridge_pert, s = "lambda.1se")
lasso_coef <- coef(lasso_pert, s = "lambda.1se")

cat("Ridge:\n")

## Ridge:
print(ridge_coef[c(target_name, "duplicate_var"), , drop = FALSE])

## 2 x 1 sparse Matrix of class "dgCMatrix"
##           lambda.1se
## lcavol      0.1560755
## duplicate_var 0.1559359

cat("\nLasso:\n")

##
## Lasso:
print(lasso_coef[c(target_name, "duplicate_var"), , drop = FALSE])

## 2 x 1 sparse Matrix of class "dgCMatrix"
##           lambda.1se
## lcavol      0.4917684
## duplicate_var .

```

2.5.2 Observed result and explanation

The perturbation confirms the expected behaviour of the two regularization methods.

Ridge assigns almost identical coefficients to the original predictor (`lcavol` = 0.1560) and its duplicate (0.1561), indicating that the effect is shared between highly correlated predictors. This behaviour improves coefficient stability in the presence of multicollinearity.

In contrast, Lasso retains the original predictor (`lcavol` = 0.4918) while shrinking the duplicated predictor exactly to zero. This demonstrates that Lasso performs variable selection by choosing one representative predictor and discarding redundant correlated variables.

Overall, the perturbation illustrates the different mechanisms of Ridge and Lasso. Ridge stabilizes coefficient estimates by sharing information across correlated predictors, whereas Lasso produces a sparse and more interpretable model by selecting only one predictor from a correlated group.

3 Problem 3. Mathematical mechanisms

3.1 3A. Ridge and conditioning

Write the ridge derivation. Define dimensions and explain uniqueness.

Let $X \in \mathbb{R}^{n \times p}$, $y \in \mathbb{R}^n$, $b \in \mathbb{R}^p$ and define

$$G = \frac{1}{n}X^\top X, \quad z = \frac{1}{n}X^\top y$$

consequently, $G \in \mathbb{R}^{p \times p}$ and $z \in \mathbb{R}^p$.

From the objective function

$$Q_\lambda(b) = \frac{1}{2n}\|y - Xb\|_2^2 + \frac{\lambda}{2}\|b\|_2^2$$

We expand Ridge objective function

$$Q_\lambda(b) = \frac{1}{2n}\|y - Xb\|_2^2 + \frac{\lambda}{2}\|b\|_2^2, \quad (\lambda > 0) \quad (1)$$

$$= \frac{1}{2n}(y - Xb)^\top (y - Xb) + \frac{\lambda}{2}b^\top b \quad (2)$$

$$= \frac{1}{2n}(y^\top y - y^\top Xb - b^\top X^\top y + b^\top X^\top Xb) + \frac{\lambda}{2}b^\top b \quad (3)$$

$$= \frac{1}{2n}(y^\top y - 2y^\top Xb + b^\top X^\top Xb) + \frac{\lambda}{2}b^\top b \quad (4)$$

$$(5)$$

We take the gradient with respect to b and substitute

$$\nabla_b Q_\lambda(b) = 0 - \frac{1}{n}y^\top X + \frac{1}{n}X^\top Xb + \lambda b \quad (6)$$

$$= -z + Gb + \lambda b \quad (7)$$

Set $\nabla_b Q_\lambda(b) = 0 \Leftrightarrow (G + \lambda I_p)b = z$ which is the **normal equation**.

Because $G = \frac{1}{n}X^\top X$ is a positive semi-definite Gram matrix, all its eigenvalues are ≥ 0 . Adding a penalty $\lambda > 0$ along the diagonal (i.e., λI_p) shifts all eigenvalues up by λ . Thus, the matrix $(G + \lambda I_p)$ is strictly positive definite (all eigenvalues $\geq \lambda > 0$), making it full rank and invertible. This guarantees a unique minimizer

$$\hat{b}_\lambda^{\text{ridge}} = (G + \lambda I_p)^{-1}z$$

```
# Use x_train and the fitted ridge lambda.min.
# safe_condition_numbers(x_train, lambda = ...)

condition_numbers <- safe_condition_numbers(x_train, ridge_model$lambda.min)
kappa_G <- condition_numbers["gram"]
kappa_ride <- condition_numbers["ridge"]

cat("Condition number of G (kappa_G):", kappa_G, "\n")
cat("Condition number of Ridge (kappa_ride):", kappa_ride, "\n")
```

3.1.1 Numerical interpretation

The OLS estimator is

$$\hat{b} = (X^\top X)^{-1} X^\top y,$$

so its numerical stability depends on the Gram matrix

$$G = X^\top X.$$

When predictors are highly correlated, the columns of X become nearly linearly dependent, making

$$\det(G) \approx 0.$$

Equivalently,

$$G = Q\Lambda Q^\top, \quad \Lambda = \text{diag}(\mu_1, \dots, \mu_p),$$

contains one or more very small eigenvalues. Since

$$G^{-1} = Q\Lambda^{-1}Q^\top, \quad \Lambda^{-1} = \text{diag}\left(\frac{1}{\mu_1}, \dots, \frac{1}{\mu_p}\right),$$

a small eigenvalue

$$\mu_{\min} \approx 0$$

produces a very large reciprocal

$$\frac{1}{\mu_{\min}} \rightarrow \infty.$$

Therefore, the inverse $(X^\top X)^{-1}$ amplifies small perturbations in the data, making the coefficient estimate

$$\hat{b} = (X^\top X)^{-1} X^\top y$$

highly sensitive to sampling variation and numerical error.

This instability is summarized by the spectral condition number

$$\kappa_2(G) = \frac{\mu_{\max}}{\mu_{\min}},$$

where a large value indicates an ill-conditioned matrix.

Ridge replaces G by

$$G + \lambda I,$$

which shifts every eigenvalue by

$$\mu_i \longrightarrow \mu_i + \lambda.$$

Hence,

$$\kappa_2(G + \lambda I) = \frac{\mu_{\max} + \lambda}{\mu_{\min} + \lambda}.$$

Since the smallest eigenvalue increases the most in relative terms, the condition number decreases, making the matrix inversion and coefficient estimates substantially more stable.

3.1.2 Geometric interpretation

The least-squares objective is

$$RSS(b) = \|y - Xb\|^2,$$

whose Hessian is

$$\nabla^2 RSS(b) = 2X^\top X = 2G.$$

Thus, the eigenvalues of G determine the curvature of the loss surface. When one eigenvalue is much smaller than the others, the curvature in that direction is very weak, producing long, narrow elliptical contours. Along this nearly flat valley, many coefficient vectors produce almost identical values of $RSS(b)$, so a small perturbation in the data can move the optimum a large distance, leading to unstable coefficients.

Ridge adds the penalty

$$\lambda \|b\|_2^2,$$

whose Hessian is

$$2\lambda I.$$

Therefore, the penalized Hessian becomes

$$2(G + \lambda I),$$

increasing the curvature in every direction, especially those corresponding to the smallest eigenvalues. The elongated ellipses become more circular, the optimum is better localized, and the coefficient estimates become more stable while retaining all predictors.

3.2 3B. Lasso optimality and soft thresholding

Derive the zero/nonzero optimality conditions and the orthonormal-design formula.

Let $X \in \mathbb{R}^{n \times p}$, $y \in \mathbb{R}^n$, $b \in \mathbb{R}^p$ and define

$$G = \frac{1}{n} X^\top X = I_p, \quad z = \frac{1}{n} X^\top y$$

consequently, $G \in \mathbb{R}^{p \times p}$ and $z \in \mathbb{R}^p$.

The objective function

$$R_\lambda(b) = \frac{1}{2n} \|y - Xb\|_2^2 + \lambda \|b\|_1, \quad \|b\|_1 = \sum_{j=1}^p |b_j|, \lambda > 0$$

We expand Lasso objective function

$$\begin{aligned} R_\lambda(b) &= \frac{1}{2n} \|y - Xb\|_2^2 + \lambda \|b\|_1 \\ &= \frac{1}{2n} (y - Xb)^\top (y - Xb) + \lambda \sum_{j=1}^p |b_j| \\ &= \frac{1}{2n} (y^\top y - y^\top Xb - b^\top X^\top y + b^\top X^\top Xb) + \lambda \sum_{j=1}^p |b_j| \end{aligned}$$

The term $y^\top Xb$ is a scalar, therefore $(y^\top Xb)^\top = b^\top X^\top (y^\top)^\top = b^\top X^\top y$.

Then the objective becomes

$$\begin{aligned} R_\lambda(b) &= \frac{1}{2n} y^\top y - \frac{1}{n} b^\top X^\top y + \frac{1}{2n} b^\top X^\top Xb + \lambda \sum_{j=1}^p |b_j| \\ &= \frac{1}{2n} y^\top y - b^\top z + \frac{1}{2} b^\top b + \lambda \sum_{j=1}^p |b_j| \\ &= \frac{1}{2n} y^\top y + \sum_{j=1}^p \left(-b_j z_j + \frac{1}{2} b_j^2 + \lambda |b_j| \right) \end{aligned}$$

The term $\frac{1}{2n} y^\top y$ is a constant with respect to the variable b , leaving a sum of p independent 1-dimensional optimization problems.

Consider a fixed j , and let $l_j(t) = \frac{1}{2} t^2 - t z_j + \lambda |t|$, for $t \in \mathbb{R}$. We analyze this in three cases because $|t|$ is *non-differentiable* at $t = 0$.

- **Case 1:** If $t > 0$, then $|t| = t$ and $l_j(t) = \frac{1}{2} t^2 - t z_j + \lambda t$. Thus, $l'_j(t) = t - z_j + \lambda$.

$$l'_j(t) = 0 \iff t = z_j - \lambda \quad (1)$$

Moreover, since we assumed $t > 0$, equation (1) is a valid minimum if and only if $z_j > \lambda$.

- **Case 2:** If $t < 0$, then $|t| = -t$ and $l_j(t) = \frac{1}{2}t^2 - tz_j - \lambda t$. Thus, $l'_j(t) = t - z_j - \lambda$.

$$l'_j(t) = 0 \iff t = z_j + \lambda \quad (2)$$

Moreover, since we assumed $t < 0$, equation (2) is a valid minimum if and only if $z_j < -\lambda$.

- **Case 3:** If $|z_j| \leq \lambda$, there is no valid solution where the derivative equals zero (neither Case 1 nor Case 2). Because the function is strictly convex, the global minimum must exist, which means it is trapped exactly at the non-differentiable corner: $t = 0$. We verify that $t = 0$ minimizes $l_j(t)$ (3).

For any $t \in \mathbb{R}$

$$\begin{aligned} l_j(t) - l_j(0) &= \frac{1}{2}t^2 - tz_j + \lambda|t| \\ &\geq \frac{1}{2}t^2 - |t||z_j| + \lambda|t| \quad (-tz_j \geq -|t||z_j|) \\ &\geq \frac{1}{2}t^2 - |t|(|z_j| - \lambda) \\ &\geq 0 \quad (|z_j| - \lambda \leq 0) \end{aligned}$$

Therefore, $l_j(t) \geq l_j(0)$ for all $t \in \mathbb{R}$, proving $t = 0$ is the global minimum for this range.

Combining these cases yields the soft-thresholding estimator for $\hat{b}_j$:

$$\hat{b}_j = \begin{cases} z_j - \lambda & \text{if } z_j > \lambda, \\ 0 & \text{if } |z_j| \leq \lambda, \\ z_j + \lambda & \text{if } z_j < -\lambda \end{cases}$$

The uniqueness is guaranteed because the objective function is strictly convex. Under the orthogonal design, the variables are perfectly independent, forming a strictly convex “bowl” shape. Even with the sharp point added by the L_1 penalty, the strict convexity mathematically ensures there is exactly one absolute global minimum.

Soft-thresholding operator under orthonormal design Assume that $\frac{1}{n}X^T X = I_p$. Let $z = \frac{1}{n}X^T y$. Then the Lasso minimizer of $R_\lambda(b)$ is *unique* and has coordinates

$$\hat{b}_j = S_\lambda(z_j), \quad \text{where the soft-thresholding function is } S_\lambda(z) = \text{sgn}(z)(|z| - \lambda)_+$$

where $(u)_+ = \max(u, 0)$.

Why lasso can produce exact zeros while ridge generally does not. Under $\frac{1}{n}X^T X = I_p$, we can compare the closed-form estimators

- **Ridge:** $\hat{b}_j^{\text{Ridge}} = (1 + \lambda)^{-1} z_j$ (proportional shrinkage). It divides the coefficient by a constant. The coefficients get extremely small, but they never become zero. The optimality condition means b_j only equals exactly zero if z_j is exactly zero (which is highly improbable with real data).
- **Lasso:** $\hat{b}_j^{\text{Lasso}} = \text{sgn}(z_j)(|z_j| - \lambda)_+$ (absolute shrinkage). It subtracts a constant from the magnitude. If a coefficient is originally small, the penalty forces it to exactly zero. Case 3 shows that Lasso maps a whole range of values to exactly zero, creating sparsity. This gives Lasso its **feature selection** capability.

3.3 3C. Connect algebra to evidence

The ridge results in Section 2B provide direct evidence for the conditioning analysis in Section 3A. At the selected tuning parameter,

$$\lambda_{\min} = 0.28468,$$

the condition number decreased from

$$\kappa_2(G) = 16.97$$

to

$$\kappa_2(G + \lambda I) = 7.49,$$

representing a reduction of more than 50%. This confirms that adding λI increased the smallest eigenvalues of the Gram matrix, making the linear system substantially better conditioned.

The improved conditioning is reflected in the fitted ridge coefficients. Compared with OLS, the coefficient of `lcavol` was shrunk from **0.6740** to **0.4587**, while the coefficient of the correlated predictor `lcp` changed from **0.0924** to **0.0824**. Rather than allowing these correlated predictors to compete for unstable coefficients, Ridge shrinks them simultaneously and distributes their shared contribution more evenly. This behaviour is also visible in the smooth coefficient paths, where all coefficients move continuously towards zero as the penalty increases, but none becomes exactly zero.

Overall, the numerical evidence supports the theoretical result of Section 3A: shifting the eigenvalues by λ improves the conditioning of the Gram matrix, leading to more stable coefficient estimates while retaining all predictors.

4 Problem 4. Elastic net and a neural-feature bridge

4.1 4A. Research question and source map

Research Question:

How does the methodological mechanism of L_2 regularization - which stabilizes models by damping parameter variance along low-eigenvalue directions - diverge from algorithmic weight decay when training deep neural networks with adaptive gradient optimizers?

Context & Methodological Bridge

In classical linear models and neural networks trained with standard Stochastic Gradient Descent (SGD), adding an L_2 penalty to the loss function and applying algorithmic weight decay are mathematically identical operations. When minimizing a loss function $\mathcal{L}(w) + \frac{\lambda}{2}\|w\|_2^2$, the derivative of the penalty introduces a $+\lambda w$ term to the gradient. Inside the standard SGD update rule, this factors into a decay multiplier: $w_{t+1} = (1 - \alpha\lambda)w_t - \alpha\nabla\mathcal{L}(w_t)$.

However, in modern deep neural networks, models are primarily trained using adaptive optimizers like Adam. Loshchilov and Hutter (2019) demonstrated that placing the L_2 penalty inside the loss function causes it to be divided by Adam’s adaptive gradient scaling factor. This neutralizes the

penalty for weights with large gradients and hyper-inflates it for sparse gradients. To restore the classical, eigenvalue-damping properties of L_2 regularization in deep learning, weight decay must be mathematically decoupled from the loss gradient (AdamW).

Source Map

Claim	Exact Source and Locator	What the Source Establishes	What it does not establish
Proposition 1: For SGD, weight decay is equivalent to L_2 regularization after learning-rate rescaling.	Loshchilov & Hutter (2019), Section 2, Proposition 1	Proves the mathematical equivalence between SGD weight decay and L_2 regularization.	Does not extend this equivalence to adaptive optimizers.
Proposition 2: For adaptive optimizers, weight decay is not equivalent to L_2 regularization.	Loshchilov & Hutter (2019), Section 2, Proposition 2	Proves adaptive gradient scaling changes the effect of L_2 regularization, motivating AdamW.	Does not prove AdamW always performs better.
Proposition 3: With a fixed preconditioner, weight decay corresponds to a weighted L_2 regularizer.	Loshchilov & Hutter (2019), Section 2, Proposition 3	Derives the equivalent weighted regularizer under fixed preconditioning.	Does not apply directly to practical Adam with a changing preconditioner.
Empirical: AdamW improves generalization and simplifies hyperparameter tuning.	Loshchilov & Hutter (2019), Sections 4.2–4.4, Figures 2–4	Shows AdamW achieves lower test error and easier hyperparameter tuning than Adam.	Does not guarantee improvements across all models or datasets.
Theoretical: Weight decay selects the minimum-norm solution, improving generalization in linear networks.	Krogh & Hertz (1991), Section 3, Eqs. (6)–(10)	Proves weight decay removes irrelevant weight components and favors the minimum-norm solution.	Does not prove the result for general non-linear networks.
Theoretical: Weight decay reduces overfitting to noisy targets.	Krogh & Hertz (1991), Section 4, Eqs. (12)–(17)	Derives an optimal weight decay for noisy linear models and shows reduced generalization error.	Does not show the same optimum applies to deep neural networks.
Theoretical: Weight decay can be extended to non-linear networks via local linearization.	Krogh & Hertz (1991), Section 5	Argues that weight decay favors lower-complexity solutions in non-linear networks.	Does not provide a rigorous proof for arbitrary non-linear networks.

Claim	Exact Source and Locator	What the Source Establishes	What it does not establish
Empirical: Weight decay lowers generalization and prediction error on NetTalk.	Krogh & Hertz (1991), Section 6, Figure 2	Demonstrates improved performance with weight decay on the NetTalk benchmark.	Does not establish the same gains for modern architectures such as CNNs or Transformers.

References

- Krogh, A., & Hertz, J. A. (1992). A simple weight decay can improve generalization. In *Advances in Neural Information Processing Systems* (Vol. 4, pp. 950-957).
- Loshchilov, I., & Hutter, F. (2019). Decoupling weight decay from gradient-based optimizers. In *International Conference on Learning Representations (ICLR)*.

4.2 4B. Elastic net on original predictors

Elastic Net combines the L_1 penalty of Lasso and the L_2 penalty of Ridge. The objective is to determine whether a mixture of the two penalties can achieve a better balance between prediction accuracy, coefficient stability, and model sparsity than either method alone.

```
alpha_grid <- c(0.10, 0.25, 0.50, 0.75, 0.90)

enet_models <- vector("list", length(alpha_grid))

enet_summary <- data.frame(
  alpha = alpha_grid,
  lambda.min = NA_real_,
  lambda.1se = NA_real_,
  cvm.min = NA_real_,
  cvm.1se = NA_real_,
  nonzero.min = NA_integer_,
  nonzero.1se = NA_integer_
)

for(i in seq_along(alpha_grid)){

  fit <- fit_cv_glmnet(
    x = x_train,
    y = y_train,
    alpha = alpha_grid[i],
    foldid = foldid
  )

  enet_models[[i]] <- fit

  id_min <- which.min(abs(fit$lambda - fit$lambda.min))
```

```

id_1se <- which.min(abs(fit$lambda - fit$lambda.1se))

enet_summary$lambda.min[i] <- fit$lambda.min
enet_summary$lambda.1se[i] <- fit$lambda.1se

enet_summary$cvm.min[i] <- fit$cvm[id_min]
enet_summary$cvm.1se[i] <- fit$cvm[id_1se]

enet_summary$nonzero.min[i] <-
  count_nonzero(fit, s = "lambda.min")

enet_summary$nonzero.1se[i] <-
  count_nonzero(fit, s = "lambda.1se")
}

knitr::kable(enet_summary, digits = 4)

```

alpha	lambda.min	lambda.1se	cvm.min	cvm.1se	nonzero.min	nonzero.1se
0.10	0.2056	0.9996	0.6363	0.7127	7	7
0.25	0.1309	0.6366	0.6346	0.7034	7	5
0.50	0.1144	0.4208	0.6335	0.7026	6	5
0.75	0.1106	0.3709	0.6331	0.7159	6	2
0.90	0.1111	0.3392	0.6323	0.7171	4	2

```

best_id <- which.min(enet_summary$cvm.min)

best_alpha <- enet_summary$alpha[best_id]
best_model <- enet_models[[best_id]]

cat("Selected alpha :", best_alpha, "\n")

## Selected alpha : 0.9
cat("Selected lambda:", best_model$lambda.min, "\n")

## Selected lambda: 0.1110642
cat("Minimum CV MSE :", min(best_model$cvm), "\n")

## Minimum CV MSE : 0.6323101
coef(best_model, s = "lambda.min")

## 9 x 1 sparse Matrix of class "dgCMatrix"
##          lambda.min
## (Intercept) 2.54491818
## lcavol      0.60062327
## lweight     0.11674208
## age         .

```

```
## lbph      .
## svi       0.18406394
## lcp       .
## gleason   .
## pgg45     0.03091681

plot(
  best_model$glmnet.fit,
  xvar = "lambda",
  label = TRUE
)

title(
  paste("Elastic Net Paths (alpha =", best_alpha, ")"),
  line = 2.5
)

abline(
  v = -log(best_model$lambda.min),
  col = "green",
  lty = 2,
  lwd = 1.5
)

abline(
  v = -log(best_model$lambda.1se),
  col = "blue",
  lty = 2,
  lwd = 1.5
)

legend(
  "topleft",
  legend = c("lambda.min", "lambda.1se"),
  col = c("green", "blue"),
  lty = 2,
  lwd = 1.5,
  bg = "white"
)
```

ASESII_24A02_ProjectExam_SampleSubmission_files/ASESII_24A02_ProjectExam_SampleSubmission_files

4.2.1 Selection rule

For each candidate value of α , an Elastic Net model was fitted using the common five-fold cross-validation assignment (`foldid`). For each model, the tuning parameter λ was selected by `lambda.min`, the value that minimized the mean cross-validation mean squared error. The final value of α was then chosen as the one producing the smallest minimum cross-validation error across the candidate grid.

The results are summarized above. As α increases from 0.10 to 0.90, the minimum cross-validation error decreases gradually from **0.6363** to **0.6323**, while the number of active predictors at `lambda.min` decreases from **7** to **4**. Consequently, $\alpha = 0.90$ was selected because it achieved the smallest cross-validation error (**CV MSE = 0.6323**) with the sparsest model among the candidate values.

At the selected tuning parameters,

- $\alpha = 0.90$,
- $\lambda_{\min} = 0.1111$,

the fitted model retained four predictors:

- `lcavol`,
- `lweight`,
- `svi`,
- `pgg45`.

The remaining four predictors (`age`, `lbph`, `lcp`, and `gleason`) were shrunk exactly to zero.

4.2.2 Relationship with Ridge and Lasso in Problem 2

Elastic Net combines the L_1 penalty of Lasso with the L_2 penalty of Ridge, producing a solution between the two methods.

In Problem 2, Ridge retained all **eight predictors** because the L_2 penalty shrinks coefficients continuously without setting them to zero, whereas Lasso selected a sparse model by shrinking several coefficients exactly to zero.

For the prostate dataset, the selected Elastic Net model uses $\alpha = 0.90$ with $\lambda_{\min} = 0.1111$ and achieves the lowest cross-validation error (**CV MSE = 0.6323**). At this solution, only four predictors (`lcavol`, `lweight`, `svi`, and `pgg45`) remain active, while the remaining four coefficients are shrunk exactly to zero.

Thus, the fitted model is much closer to Lasso in terms of variable selection, but the remaining **10%** L_2 penalty ($1 - \alpha = 0.10$) still provides additional coefficient stabilization for correlated predictors. Consequently, Elastic Net combines the sparsity of Lasso with part of the stability and grouping behaviour of Ridge.

4.3 4C. Fixed ReLU features

Using the standardized training predictors, we constructed $M = 200$ fixed random ReLU features of the form

$$h_m(x) = \max\{a_m^\top x + c_m, 0\}, \quad m = 1, \dots, M,$$

where

$$a_m \sim N\left(0, \frac{1}{p}I_p\right),$$

and

$$c_m \sim N(0, 0.5^2).$$

The random weights and offsets were generated once using the declared seed and then applied unchanged to both the training and test sets. Hidden features with zero variance in the training data were removed using training information only.

```
set.seed(seeds$features)

relu <- function(z) pmax(z, 0)

M <- 200L
p <- ncol(x_train)

# Generate random hidden-layer parameters
A_matrix <- matrix(
  rnorm(
    p * M,
    mean = 0,
    sd = sqrt(1 / p)
  ),
  nrow = p,
  ncol = M
)

c_bias <- rnorm(
  M,
  mean = 0,
  sd = 0.5
)

# Construct ReLU features
H_train_raw <- relu(
  sweep(
    x_train %*% A_matrix,
    2,
    c_bias,
    "+"
  )
)

H_test_raw <- relu(
```

```

sweep(
  x_test %*% A_matrix,
  2,
  c_bias,
  "+"
)
)

# Remove constant hidden features using training only
keep_hidden <- apply(
  H_train_raw,
  2,
  sd
) > 0

H_train_raw <- H_train_raw[
  ,
  keep_hidden,
  drop = FALSE
]

H_test_raw <- H_test_raw[
  ,
  keep_hidden,
  drop = FALSE
]

cat(
  "Hidden features retained:",
  ncol(H_train_raw),
  "out of",
  M,
  "\n"
)

## Hidden features retained: 198 out of 200

# Standardize hidden features using training statistics
h_prep <- fit_scaler(H_train_raw)

H_train <- apply_scaler(
  H_train_raw,
  h_prep
)

H_test <- apply_scaler(
  H_test_raw,
  h_prep

```

```
)
```

Ridge, lasso, and elastic net were fitted on the fixed ReLU feature matrix using the shared five-fold cross-validation assignment. For elastic net, the mixing parameter α was selected from the same candidate grid as in Part 4B.

```
ridge_relu <- fit_cv_glmnet(  
  H_train,  
  y_train,  
  alpha = 0,  
  foldid = foldid  
)  
  
lasso_relu <- fit_cv_glmnet(  
  H_train,  
  y_train,  
  alpha = 1,  
  foldid = foldid  
)  
  
alpha_grid <- c(  
  0.10,  
  0.25,  
  0.50,  
  0.75,  
  0.90  
)  
  
enet_models <- lapply(  
  alpha_grid,  
  function(alpha)  
    fit_cv_glmnet(  
      H_train,  
      y_train,  
      alpha = alpha,  
      foldid = foldid  
    )  
)  
  
cv_errors <- sapply(  
  enet_models,  
  function(model) min(model$cvm)  
)  
  
best_id <- which.min(cv_errors)  
  
best_alpha_relu <- alpha_grid[best_id]
```

```
enet_relu <- enet_models[[best_id]]
```

The selected tuning values, training cross-validation errors, and numbers of active hidden features are summarized below.

```
relu_summary <- data.frame(  
  
  Model = c(  
    "Ridge",  
    "Lasso",  
    "Elastic net"  
  ),  
  
  Alpha = c(  
    0,  
    1,  
    best_alpha_relu  
  ),  
  
  Lambda = c(  
    ridge_relu$lambda.min,  
    lasso_relu$lambda.min,  
    enet_relu$lambda.min  
  ),  
  
  CV_Error = c(  
    min(ridge_relu$cvm),  
    min(lasso_relu$cvm),  
    min(enet_relu$cvm)  
  ),  
  
  Active_Hidden = c(  
    count_nonzero(ridge_relu),  
    count_nonzero(lasso_relu),  
    count_nonzero(enet_relu)  
  )  
)  
  
knitr::kable(  
  relu_summary,  
  digits = 4,  
  caption = "Cross-validation results on the fixed ReLU features."  
)
```

Table 1: Cross-validation results on the fixed ReLU features.

Model	Alpha	Lambda	CV_Error	Active_Hidden
Ridge	0.0	7.9035	0.6980	198
Lasso	1.0	0.1414	0.7719	11
Elastic net	0.1	1.0208	0.7323	46

4.3.1 Fixed and learned parameters

The hidden-layer weight vectors a_m and offsets c_m were fixed after being generated once from their specified normal distributions. These random feature parameters were reused unchanged for both the training and test sets and were not estimated from the data.

The learned quantities were the regression coefficients associated with the hidden features and the intercept. Cross-validation selected the regularization parameter λ for ridge and lasso, and selected both the mixing parameter α and the regularization parameter λ for elastic net.

Among the three models, ridge achieved the smallest training cross-validation error (0.6980) but retained almost every hidden feature (198), producing the least sparse model. Lasso selected only 11 active hidden features, giving the sparsest representation but also the largest cross-validation error (0.7719). Elastic net selected $\alpha = 0.10$, retained 46 hidden features, and achieved an intermediate cross-validation error (0.7323), providing a compromise between prediction accuracy and sparsity. —

4D. Locked analysis and final holdout evaluation

4.3.2 Locked analysis declaration

Before inspecting the holdout responses (`y_test`), the analysis was locked as follows.

- Predictors were standardized using the training statistics only.
- The same five-fold cross-validation assignment was used for all regularized models.
- Fixed ReLU features were generated once using 240406 with $M = 200$ hidden features and reused unchanged.
- Candidate models:
 - OLS
 - Ridge (`lambda.min`)
 - Lasso (`lambda.min`)
 - Elastic Net (`alpha = 0.90, lambda.min`)
 - Ridge + ReLU (`lambda.min`)
 - Lasso + ReLU (`lambda.min`)
 - Elastic Net + ReLU (`alpha = 0.10, lambda.min`)
- The nominated deployment model was **Elastic Net on the original predictors**.
- Holdout performance was evaluated using RMSE and MAE.

----- Original-feature models -----

```
pred_ols <- predict(
  ols_model,
  newdata = as.data.frame(x_test)
)
```

```

pred_ridge <- as.numeric(
  predict(ridge_model, newx = x_test, s = "lambda.min")
)

pred_lasso <- as.numeric(
  predict(lasso_model, newx = x_test, s = "lambda.min")
)

pred_enet <- as.numeric(
  predict(best_model, newx = x_test, s = "lambda.min")
)

## ----- ReLU-feature models -----

pred_ridge_relu <- as.numeric(
  predict(ridge_relu, newx = H_test, s = "lambda.min")
)

pred_lasso_relu <- as.numeric(
  predict(lasso_relu, newx = H_test, s = "lambda.min")
)

pred_enet_relu <- as.numeric(
  predict(enet_relu, newx = H_test, s = "lambda.min")
)

holdout_results <- data.frame(
  Model = c(
    "OLS",
    "Ridge",
    "Lasso",
    "Elastic Net",
    "Ridge + ReLU",
    "Lasso + ReLU",
    "Elastic Net + ReLU"
  ),
  RMSE = c(
    score_regression(y_test, pred_ols)["RMSE"],
    score_regression(y_test, pred_ridge)["RMSE"],
    score_regression(y_test, pred_lasso)["RMSE"],
    score_regression(y_test, pred_enet)["RMSE"],
    score_regression(y_test, pred_ridge_relu)["RMSE"],
    score_regression(y_test, pred_lasso_relu)["RMSE"],
    score_regression(y_test, pred_enet_relu)["RMSE"]
  ),
  MAE = c(
    score_regression(y_test, pred_ols)["MAE"],

```

```

    score_regression(y_test, pred_ridge) ["MAE"],
    score_regression(y_test, pred_lasso) ["MAE"],
    score_regression(y_test, pred_enet) ["MAE"],
    score_regression(y_test, pred_ridge_relu) ["MAE"],
    score_regression(y_test, pred_lasso_relu) ["MAE"],
    score_regression(y_test, pred_enet_relu) ["MAE"]
  )
)

knitr::kable(
  holdout_results,
  digits = 4,
  caption = "Holdout performance of all locked candidate models."
)

```

Table 2: Holdout performance of all locked candidate models.

	Model	RMSE	MAE
	OLS	0.7296	0.6137
	Ridge	0.7654	0.6230
	Lasso	0.7743	0.6167
	Elastic Net	0.7676	0.6137
	Ridge + ReLU	0.7660	0.6238
	Lasso + ReLU	0.7787	0.6391
	Elastic Net + ReLU	0.7563	0.6134

4.3.3 Holdout evaluation

The holdout evaluation was performed only once after the analysis had been locked. No hyperparameters were retuned, no predictors were replaced, and no random features were regenerated after inspecting the holdout responses.

OLS achieved the smallest holdout RMSE (0.7296), while Elastic Net with fixed ReLU features achieved the smallest MAE (0.6134). Among the regularized models, Elastic Net with fixed ReLU features also produced the smallest RMSE (0.7563), improving on Elastic Net fitted to the original predictors (0.7676).

The holdout evidence does not fully support the training-based decision. Elastic Net on the original predictors was nominated using cross-validation, but OLS performed best on this particular holdout sample. Since the holdout set contains only 20 observations, this difference is reasonably attributed to sampling variability. Following the locked-analysis protocol, the nominated deployment model is not changed after viewing the holdout results.

4.4 4E. Deep-learning connection and limitations

4.5 4E. Discussion

4.5.1 1. L_2 penalty versus weight decay

In this experiment, only the **output weights** are optimized while the hidden-layer parameters remain fixed. Applying an L_2 penalty to the output weights is therefore equivalent to **ridge regression**, which shrinks the learned coefficients to reduce overfitting. However, in fully trainable neural networks using adaptive optimizers such as Adam, adding an L_2 penalty to the loss is generally **not equivalent** to weight decay because adaptive gradient scaling changes the effect of the regularization term. AdamW resolves this by decoupling weight decay from the gradient update (?).

4.5.2 2. Zero output weights do not create sparse hidden weights

Each hidden feature is generated from fixed random input-to-hidden weights and remains unchanged throughout training. If an output weight becomes zero, the corresponding hidden feature no longer contributes to the prediction. However, the input-to-hidden weights that generate this feature remain unchanged because they are never optimized. Therefore, sparsity occurs only in the **output layer**, not in the hidden layer or the network structure.

4.5.3 3. Dropout is not the same as pruning

Dropout randomly masks a subset of hidden activations during each training iteration, with a new mask sampled for every mini-batch. During inference, all neurons remain active. In contrast, pruning permanently removes weights or neurons from the network. Therefore, dropout is a temporary regularization method, whereas pruning permanently changes the model architecture.

4.5.4 4. Conclusions cannot be generalized to fully trained deep networks

This experiment uses a **frozen hidden layer**, where the random hidden weights and biases are generated once and remain fixed. Only the output weights are learned. Consequently, the experiment evaluates regularization on **fixed random features** rather than on representations learned jointly with the hidden layers. The results therefore cannot be directly generalized to fully trained deep neural networks, where all layers are optimized together (?).

4.5.5 Limitations

1. **Sensitivity to the random-feature seed.** The hidden features are generated from a single random initialization. Different seeds may produce different feature representations and model performance, so the results may depend on the chosen seed.
2. **Frozen hidden representation.** Since only the output layer is trained, the experiment does not evaluate how regularization affects feature learning in end-to-end trained neural networks.
3. **Limited experimental setting.** The conclusions are based on one hidden-layer architecture with 200 ReLU features and a single dataset. They may not generalize to deeper architectures, different activation functions, or other datasets.

Abstract

This study investigates log prostate-specific antigen (`lpsa`) prediction in 97 prostatectomy patients using clinical covariates. To mitigate multicollinearity and prevent target leakage, we implement an immutable 80/20 train/test partition with training-only preprocessing. We compare Ordinary Least Squares (OLS) against regularized linear models (Ridge, Lasso, Elastic Net) and a non-linear Random ReLU feature representation ($M = 200$). Mathematical derivations confirm that L_2 regularization stabilizes ill-conditioned Gram matrices by upwardly shifting eigenvalues, reducing the condition number by over 50% (κ dropped from 16.97 to 7.49). Empirical five-fold cross-validation identifies Lasso (`lambda.min`) and Elastic Net as superior predictors over baseline OLS and static non-linear projections. For clinical deployment, we nominate the Elastic Net model on original predictors, which optimizes out-of-sample generalization while collaboratively shrinking correlated biomarkers. We emphasize that regularized coefficients represent conditional predictive slopes rather than causal biological mechanisms.

5 Problem 5. Report quality and reproducibility

5.1 5A. Reproduction record

To independently reproduce the results, figures, and evaluation tables in this report, execute the following steps from a clean R session:

5.1.1 1. Folder Layout and Setup

Ensure the working directory contains the R Markdown file, the bibliography file, and the raw CSV datasets inside a local `data/` subdirectory:

```
ASESII_24A02_Project/  
  ASESII_24A02_ProjectExam_Ridge_Lasso_ProblemStatement.pdf  
  data/  
    prostate.csv  
  README.md  
  templates/  
    ASESII_24A02_ProjectExam_SampleSubmission.Rmd  
    ASESII_24A02_ProjectExam_SampleSubmission.pdf  
    Group06_AI_Log.csv  
    references.bib
```

5.1.2 2. Execution Commands

Open a clean R session (in RStudio via `Session -> Restart R`), clear the workspace, and render the document from the console using relative paths:

```
# Clear workspace and verify clean session  
rm(list = ls())  
gc()  
  
# Verify required packages are installed  
required_pkgs <- c("tidyverse", "glmnet", "broom", "knitr", "rmarkdown")  
invisible(lapply(required_pkgs, function(pkg) {
```

```

if (!requireNamespace(pkg, quietly = TRUE)) install.packages(pkg)
}))

# Render the report to HTML/PDF using relative paths
rmarkdown::render("templates/ASESII_24A02_ProjectExam_SampleSubmission.Rmd", output_format = "

```

5.1.3 3. Holdout Quarantine Audit

- **Where holdout responses first enter:** The holdout response vector `y_test` is partitioned during the initial data split in **Section 1B (split-and-scale chunk)**, but it is strictly quarantined throughout all exploratory data analysis (Section 1C), baseline OLS modeling (Section 2A), regularized model tuning (Sections 2B–2C), and non-linear feature engineering (Section 4C).
- **Point of first computation:** `y_test` enters mathematical computation for the absolute first and only time in **Section 4D (final-holdout-evaluation chunk)**, where it is evaluated against fixed candidate predictions to compute out-of-sample Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE).

5.2 5B. Recommendation and limitations

5.2.1 1. Final deployment recommendation

Although the ordinary least squares (OLS) model achieved the lowest holdout RMSE on this particular train/test split, we recommend the **Elastic Net model** (using the tuning parameters selected by cross-validation) for practical deployment.

This recommendation reflects the model selection procedure adopted in this study. Hyperparameters were selected exclusively by cross-validation on the training data, while the holdout set was reserved for a single unbiased final evaluation. Because the prostate cancer predictors exhibit moderate multicollinearity, Elastic Net provides a good balance between predictive performance, coefficient stability, and model complexity by combining Ridge shrinkage with Lasso variable selection. Although OLS performed slightly better on this particular holdout split, Elastic Net is expected to provide more robust performance across future samples.

5.2.2 2. Prediction versus interpretation

The preferred model depends on the objective of the analysis.

- **Prediction.** For predicting future patient outcomes, the selected Elastic Net model provides a good compromise between predictive accuracy, coefficient stability, and model complexity. Its regression coefficients should be interpreted only as conditional associations within the fitted model and **not as evidence of causal effects**.
- **Interpretation.** If the primary objective is identifying a compact subset of important predictors rather than maximizing predictive accuracy, a more strongly regularized Lasso model (for example, using `lambda.1se`) may be preferred. This produces a simpler and more interpretable model while accepting a small reduction in predictive performance.

5.2.3 3. Study limitations

- **Small sample size.** The analysis is based on only 97 observations (77 training and 20 holdout samples). Consequently, performance estimates are subject to sampling variability, and different data splits may produce different model rankings.
- **Limited external validity.** The dataset represents a historical clinical cohort from a single institution. External validation on more recent and diverse patient populations would be required before clinical deployment.
- **Single holdout evaluation.** Model performance was evaluated using one fixed holdout partition. Although this preserves an unbiased final assessment, it does not quantify the variability of the reported test errors across repeated train/test splits.
- **Fixed random feature mapping.** The ReLU feature expansion in Section 4 used randomly generated hidden features without backpropagation. Therefore, the results evaluate fixed random feature representations rather than fully trained neural networks.

5.3 5C. AI verification record

AI-assisted item	Verification procedure	Evidence	Final decision
Ridge regression derivation (Section 3A)	Re-derived the penalized least squares objective using matrix calculus and verified each differentiation step against standard references.	Confirmed the normal equation $(X^\top X + \lambda I_p)\hat{\beta} = X^\top y$ and the resulting closed-form Ridge estimator.	Accepted with minor revision: Improved notation and expanded intermediate algebraic steps for clarity.
Lasso soft-thresholding derivation (Section 3B)	Verified the coordinate-wise optimization using subgradient conditions under an orthonormal design matrix.	Confirmed that the solution is the soft-thresholding operator and that the derivation is valid only under the stated orthonormal assumption.	Accepted: Mathematical derivation was consistent after explicitly stating the required assumptions.
Manual VIF helper function	Compared the custom implementation with the output of <code>car::vif()</code> using the baseline OLS model.	Both methods produced identical variance inflation factors up to numerical precision.	Accepted: The custom implementation was retained to avoid introducing an additional package dependency.

AI-assisted item	Verification procedure	Evidence	Final decision
Discussion of weight decay and adaptive optimization (Section 4A)	Checked theoretical claims against the original literature on L2 regularization and decoupled weight decay.	Confirmed that adaptive optimizers such as Adam make L2 regularization and weight decay no longer equivalent, consistent with Loshchilov and Hutter (2019).	Accepted with revision: Removed unsupported claims and restricted the discussion to statements directly supported by the cited references.

Preflight and session information

```
# Audit: Verify every critical pipeline object exists before generating final deliverables
required_objects <- c(
  "train_id", "test_id", "x_train", "x_test", "y_train", "y_test",
  "foldid", "ols_model", "ridge_model", "lasso_model"
)

missing_objects <- setdiff(required_objects, ls())
if (length(missing_objects) > 0) {
  stop(
    "Preflight Check Failed. Missing required pipeline objects: ",
    paste(missing_objects, collapse = ", ")
  )
}

# Safeguard: Verify absolute zero row overlap between train and test partitions
stopifnot("Data Leakage Alert: Train and test indices overlap!" = length(intersect(train_id, test_id)) == 0)
cat("Preflight audit passed: Folds, objects, and leakage boundaries are verified.\n")

# Print full R engine, OS, and package version details for reproducibility auditing
sessionInfo()

## R version 4.6.0 (2026-04-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Vietnamese_Vietnam.utf8  LC_CTYPE=Vietnamese_Vietnam.utf8
## [3] LC_MONETARY=Vietnamese_Vietnam.utf8 LC_NUMERIC=C
## [5] LC_TIME=Vietnamese_Vietnam.utf8
##
## time zone: Asia/Saigon
```

```
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] knitr_1.51      broom_1.0.13   glmnet_5.0     Matrix_1.7-5
## [5] lubridate_1.9.5 forcats_1.0.1  stringr_1.6.0  dplyr_1.2.1
## [9] purrr_1.2.2     readr_2.2.0    tidyr_1.3.2    tibble_3.3.1
## [13] ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] generics_0.1.4   shape_1.4.6.1   stringi_1.8.7    lattice_0.22-9
## [5] hms_1.1.4         digest_0.6.39    magrittr_2.0.5    timechange_0.4.0
## [9] evaluate_1.0.5    grid_4.6.0       RColorBrewer_1.1-3 iterators_1.0.14
## [13] fastmap_1.2.0     foreach_1.5.2    backports_1.5.1   survival_3.8-6
## [17] scales_1.4.0      codetools_0.2-20 cli_3.6.6         rlang_1.2.0
## [21] splines_4.6.0     withr_3.0.2      yaml_2.3.12       otel_0.2.0
## [25] tools_4.6.0       tzdb_0.5.0       vctrs_0.7.3       R6_2.6.1
## [29] lifecycle_1.0.5   pkgconfig_2.0.3  pillar_1.11.1     gtable_0.3.6
## [33] glue_1.8.1        Rcpp_1.1.2       xfun_0.58         tidyselect_1.2.1
## [37] rstudioapi_0.18.0 farver_2.1.2      htmltools_0.5.9   rmarkdown_2.31
## [41] compiler_4.6.0    S7_0.2.2
```

Krogh, Anders, and John A. Hertz. 1992. “A Simple Weight Decay Can Improve Generalization.” *Advances in Neural Information Processing Systems (NeurIPS)* 4: 950–57.

Loshchilov, Ilya, and Frank Hutter. 2019. “Decoupling Weight Decay from Gradient-Based Optimizers.” *International Conference on Learning Representations (ICLR)*.